

Trabajo Práctico 3

Perceptrón simple y multicapa

Grupo 5

Federico Ruckauf

Matías Romanato

Celestino Garrós

Leo Weitz

Tres encargos de CompanyX, del perceptrón simple a la red multicapa

1

Fraude: destilar BigModel en un TinyModel

Perceptrón simple, lineal vs no lineal, que imita la probabilidad de fraude de BigModel y es mucho más barato de usar y guardar.

2

Dígitos manuscritos, del 0 al 9

Perceptrón multicapa para digitalizar los datos que hoy se cargan a mano durante la distribución.

3

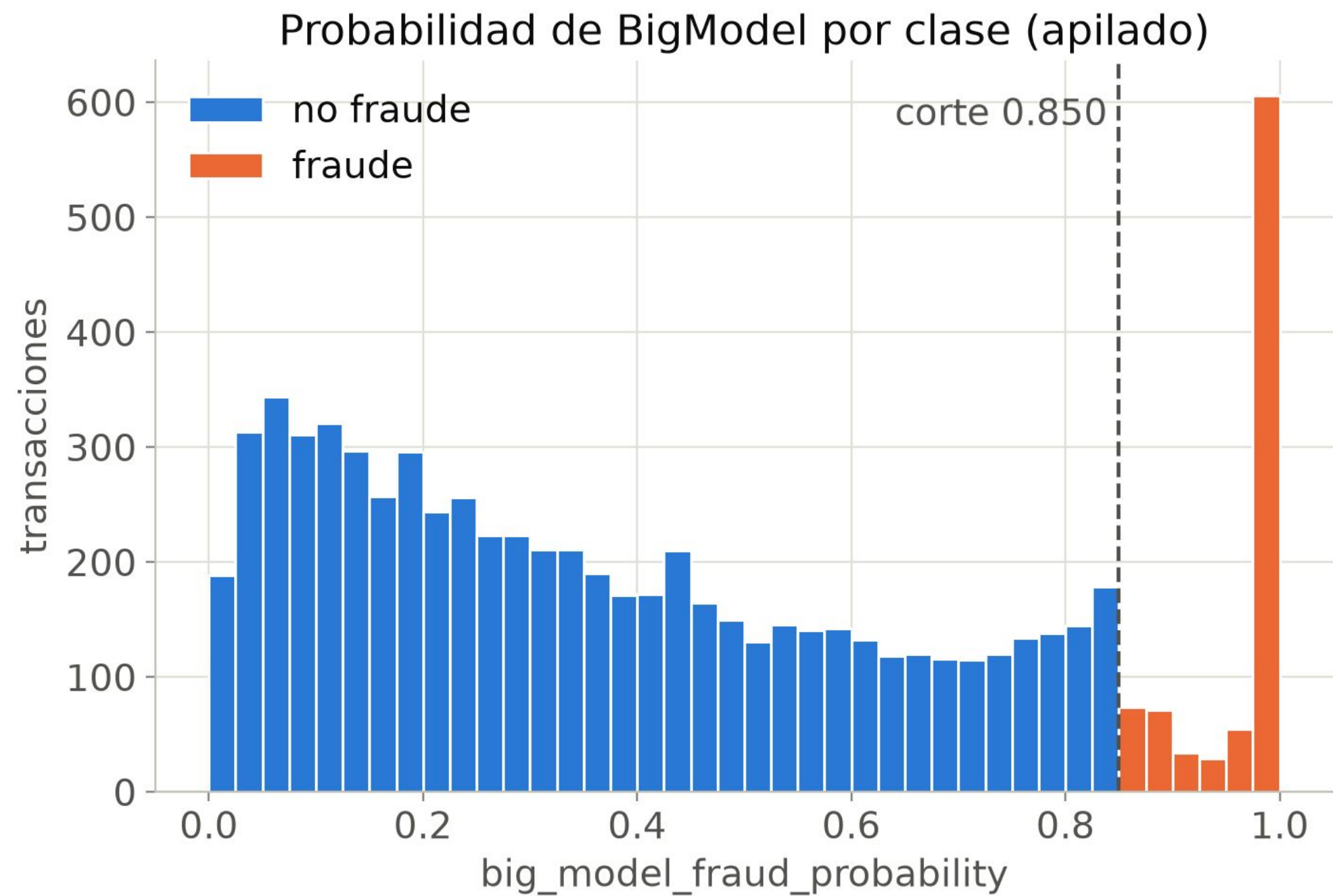
Dígitos con más datos: accuracy $\geq$ 98 %

La primera iteración no alcanzó. Con datos nuevos y otras técnicas, el cliente pide llegar al 98 %.

Perceptrón simple para detectar fraudes

Destilar BigModel en un TinyModel

¿Se puede imitar a BigModel con un modelo diminuto?



Probabilidad de BigModel en las 7500 transacciones, por clase

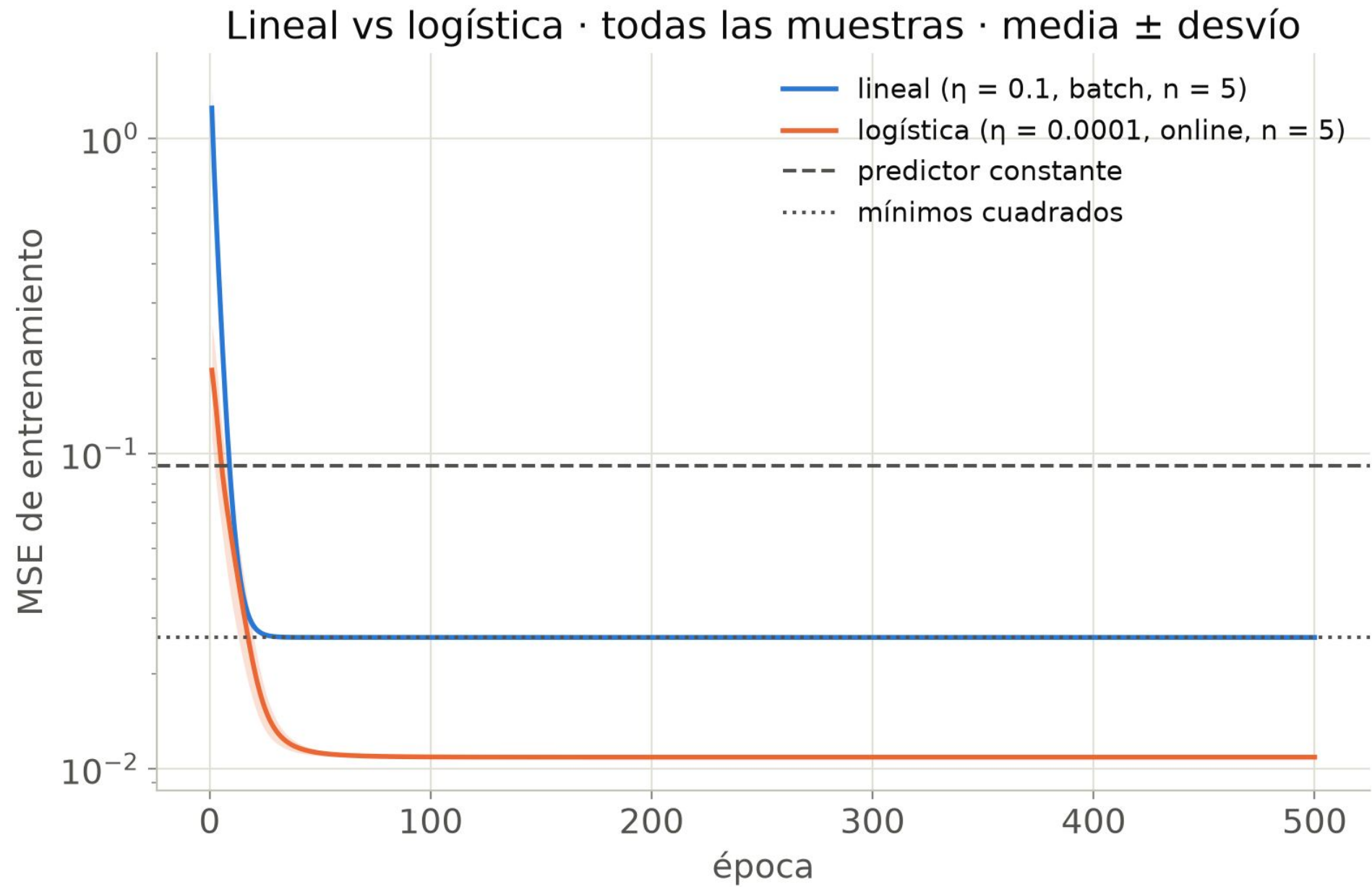
7500 9 11,6 %
transacciones columnas de fraude

El TinyModel aprende la probabilidad de BigModel (regresión, MSE).

La etiqueta de fraude solo se usa para evaluar.

Datos limpios: sin faltantes ni duplicados.
Escala de 10 a 10⁹: normalizamos con z-score.

¿Cuánto puede aprender un perceptrón simple?



0,091

constante

0,026

lineal

0,011

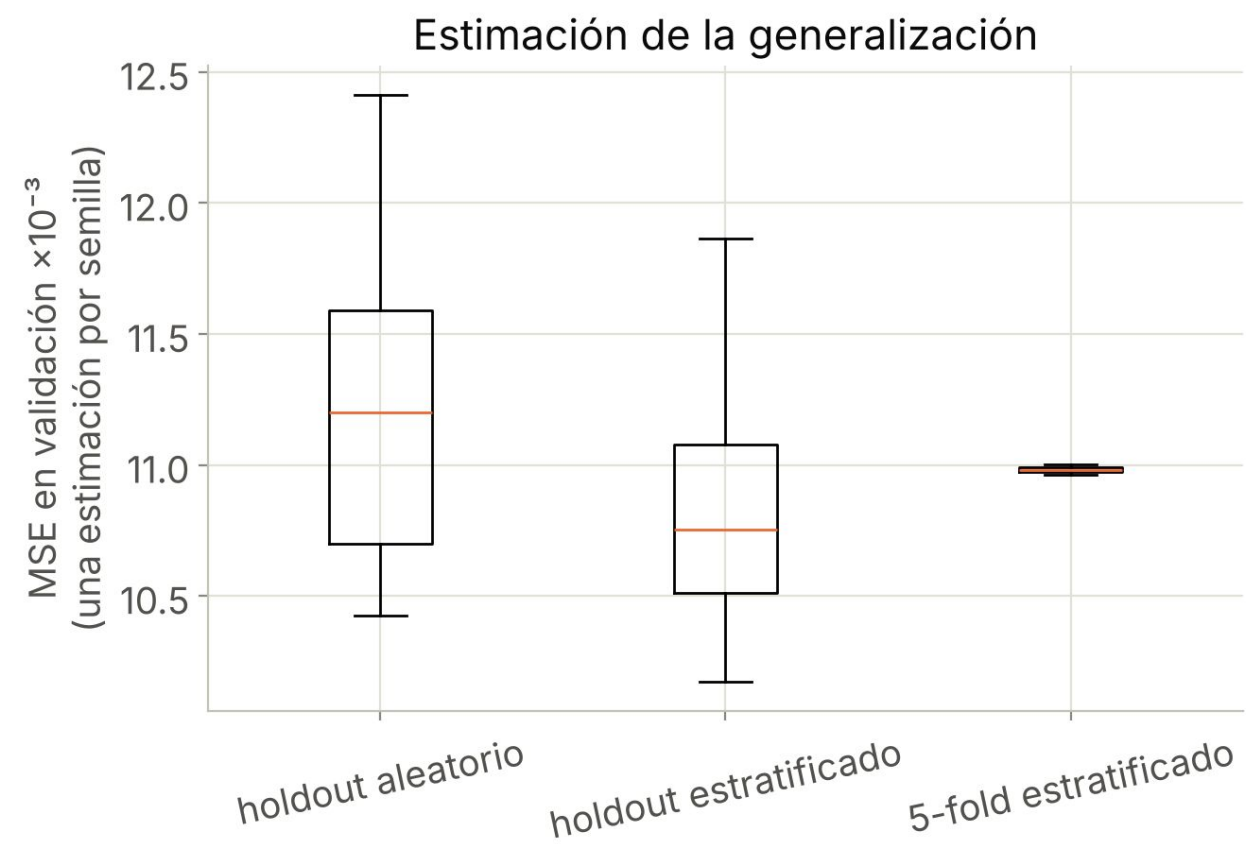
logística

error de entrenamiento (MSE)

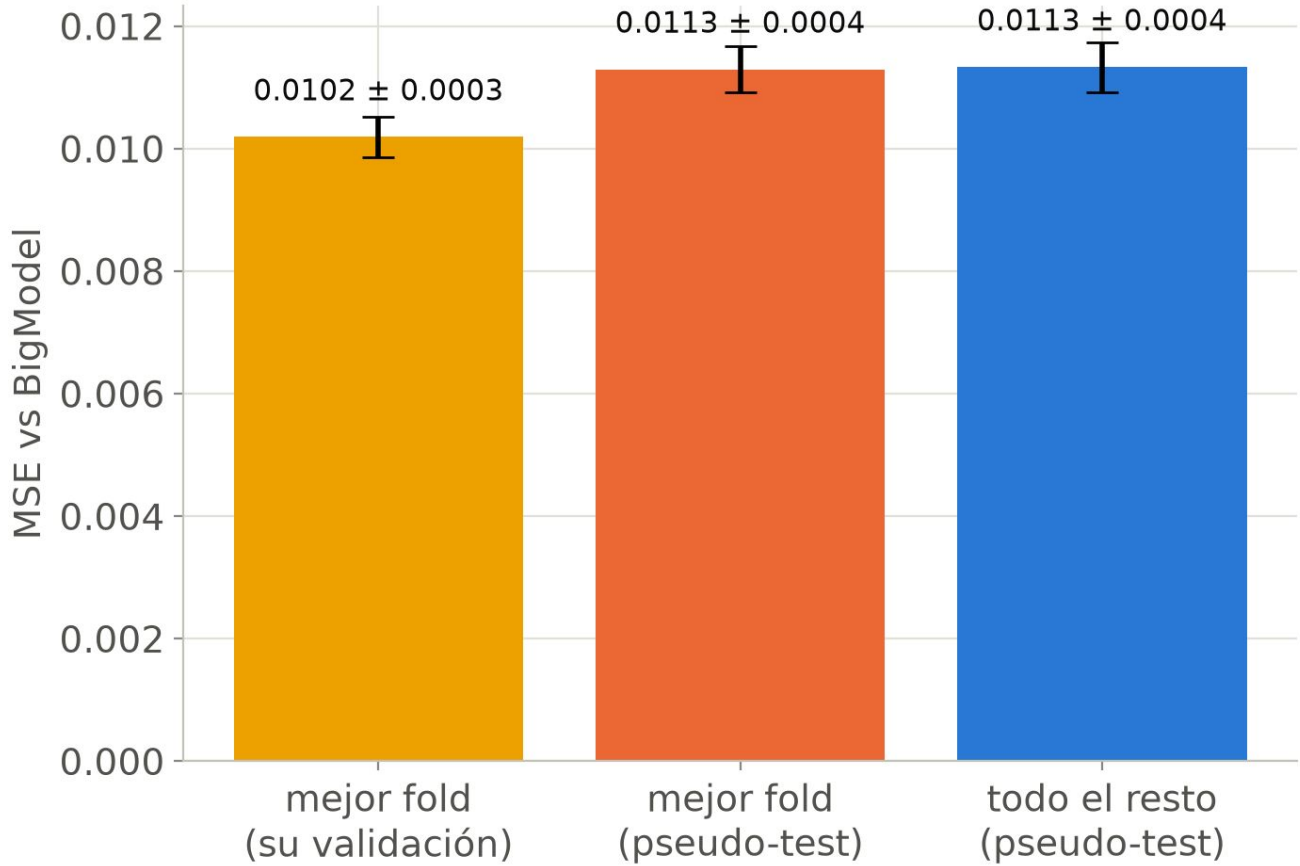
Perceptrón lineal y logístico, entrenados con todas las muestras

Barrido de tasa de aprendizaje y modo (online / batch)

¿Cómo medir si generaliza, y con qué datos entrenar?



Error estimado según la estrategia de partición · 20 semillas por estrategia



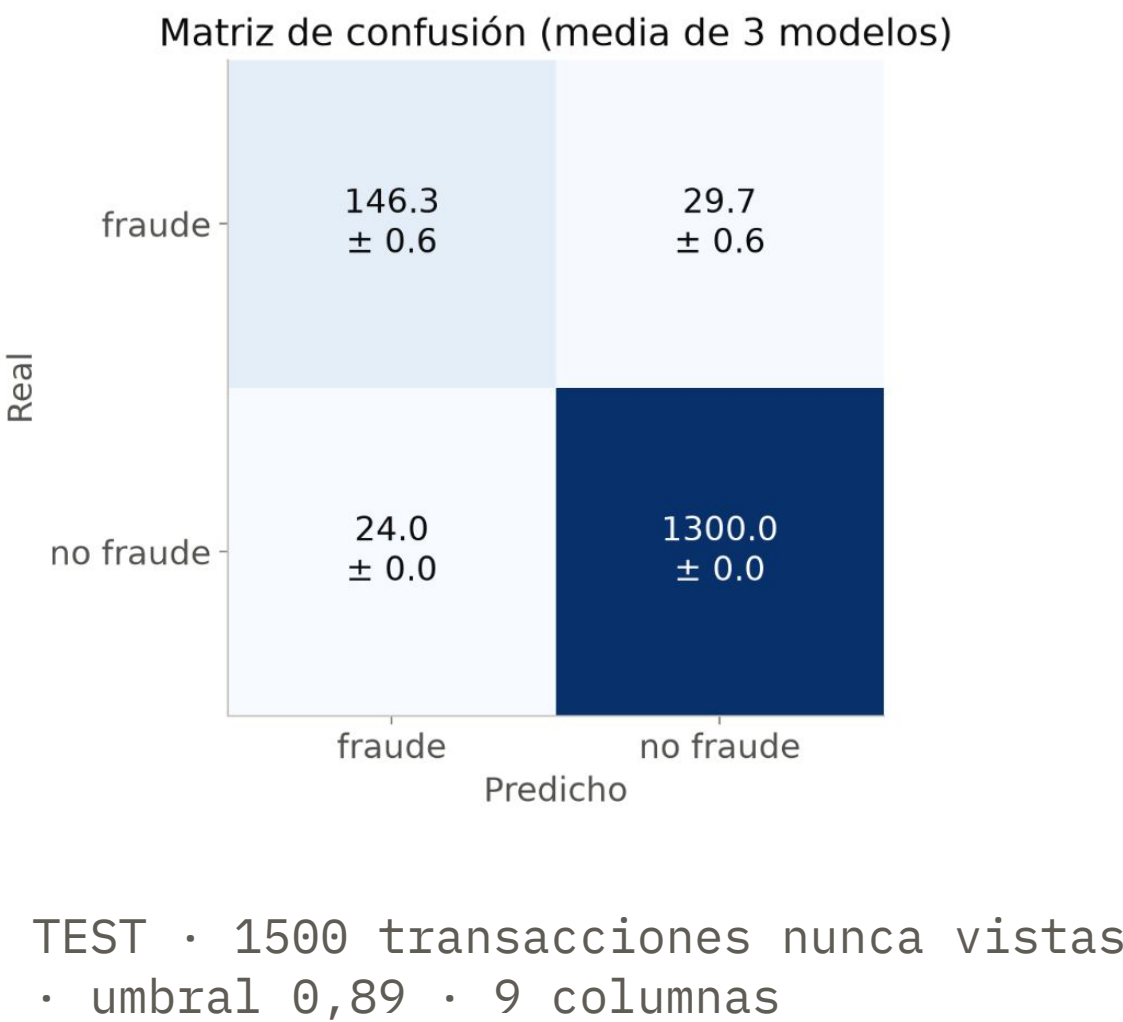
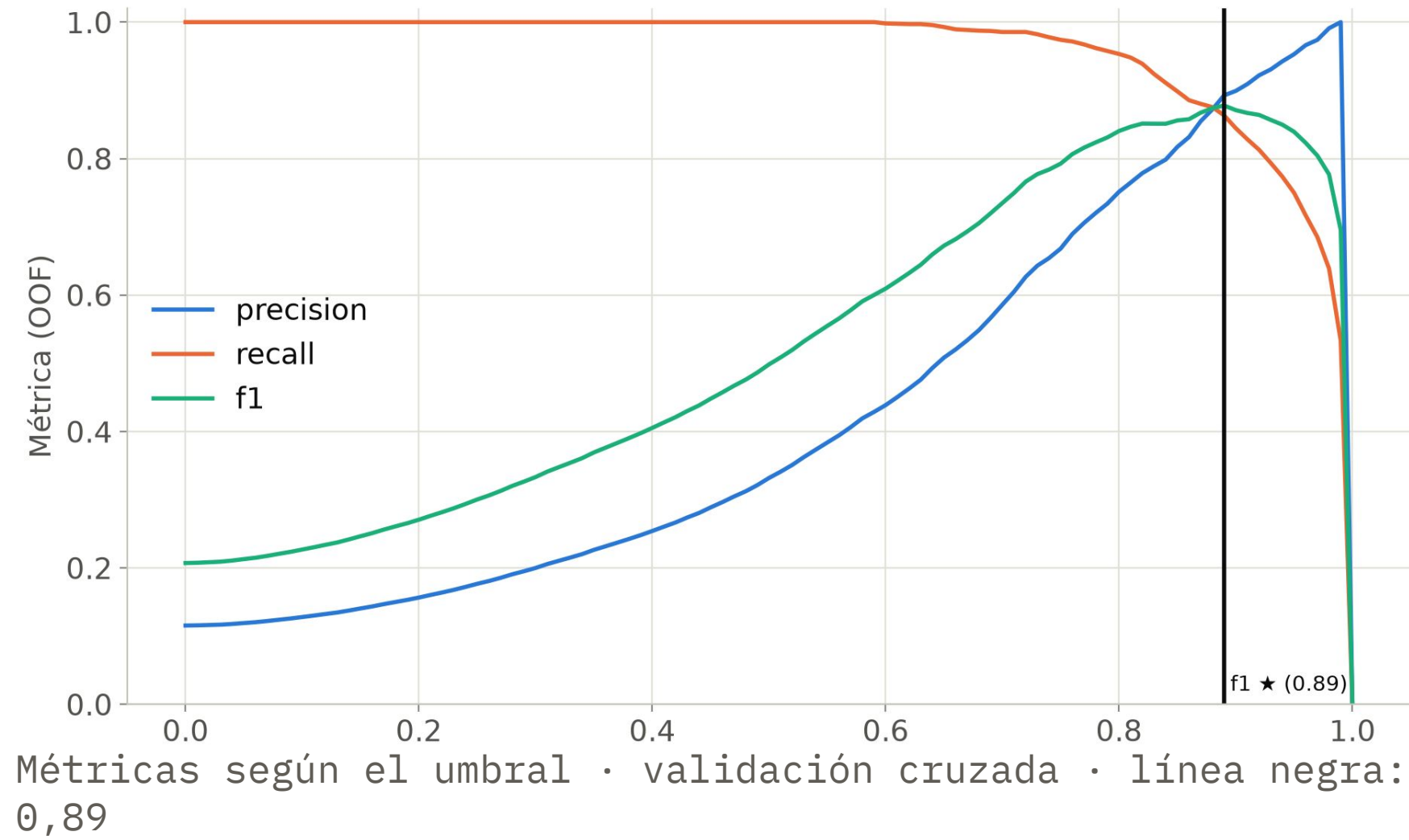
Fold con mejor puntaje vs entrenar con todo · error en datos nuevos

Partición: TEST del 20 % y 5-fold en el resto, estratificados: misma proporción de fraude en cada parte.

5-fold: la estimación casi no depende de la semilla (desvío ~45 veces menor que un 80/20).

Métricas: precision, recall y F1. La accuracy engaña: sin detectar ningún fraude da 88 %.

¿A partir de qué probabilidad hay que alertar?



Umbral elegido en validación (criterio F1)

TEST: recall 0,86 y precision 0,89.

146 de 176
fraudes detectados

¿Hacen falta las 9 columnas?

	9 columnas	6 columnas
Modelo · parámetros	10	7
Validación · error vs BigModel (MSE)	0,0110	0,0111
Validación · precision (umbral 0,81)	0,765	0,765
Validación · umbral elegido	0,81	0,81
Validación · fraudes detectados	94,8 %	94,7 %
Validación · alertas falsas cada 1000	33,7	33,7
TEST · fraudes detectados	93,8 %	91,9 %
TEST · precision	0,767	0,772

Las 3 columnas extra casi no pesan: fecha y tiempo desde el login ≈ 0 ; la resolución, 0,015 (las principales, entre 0,11 y 0,39).

Entregamos

6 columnas

7 parámetros · umbral 0,81

En TEST, cada 1000:

108 de 117

fraudes detectados

32

alertas falsas

Perceptrón Multi Capa para clasificar dígitos

Dígitos manuscritos, del 0 al 9

MLP para clasificación de dígitos

Entradas y salidas

Entrada: 28×28 píxeles en $[0, 1]$

Salida: 10 neuronas, una por dígito. La de mayor valor indica la predicción

Pérdida

MSE promediado por muestra y por salida.

Activaciones

Salida: logística, con $\beta = 1$.

Ocultas:

- Ej. 2 $\rightarrow$ **tanh**
- Ej. 3 $\rightarrow$ **ReLU**

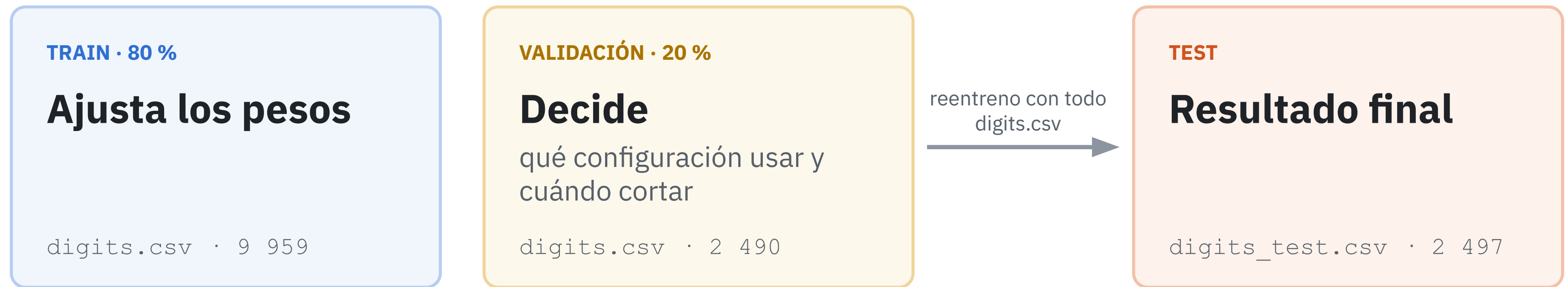
Inicialización y entrenamiento

Inicializado con pesos aleatorios chicos.

- **Xavier** para tanh
- **He** para ReLU

Entrenamiento con mini-batch de 32

Partición de Train y Validation y Métricas



Métricas

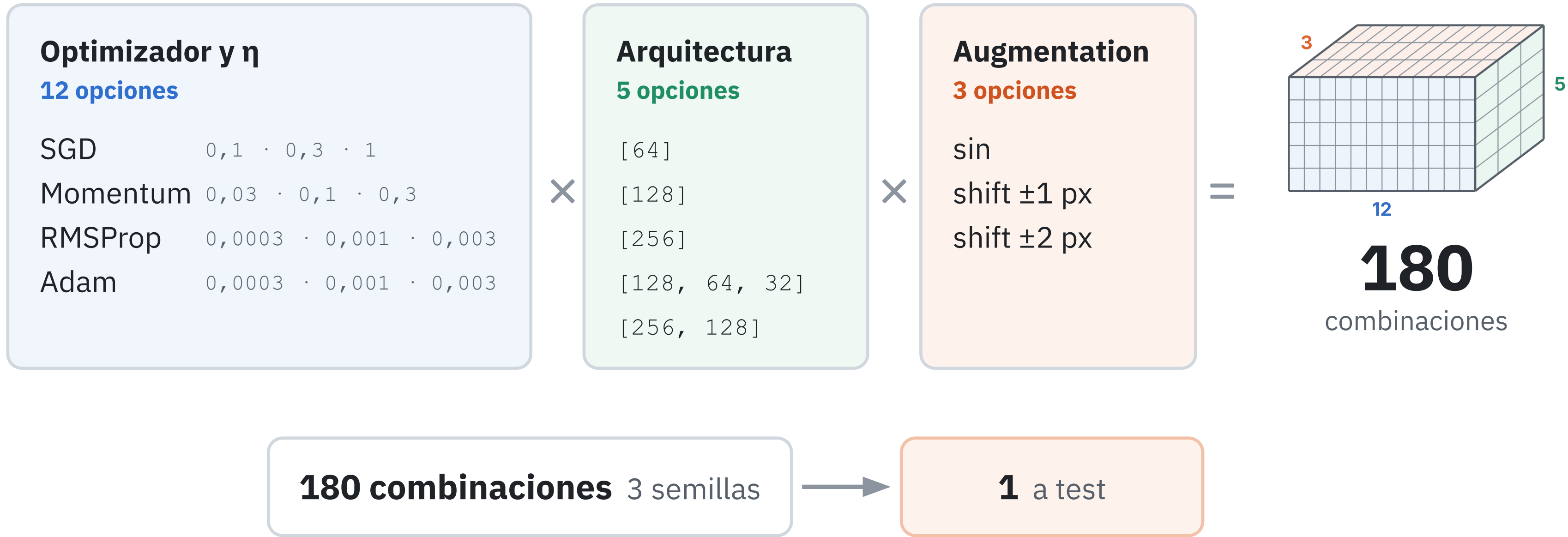
- **Accuracy**
- **Matriz de confusión y recall** por dígito

Partición estratificada

El 5 tiene 271 muestras y no hay ningún 8

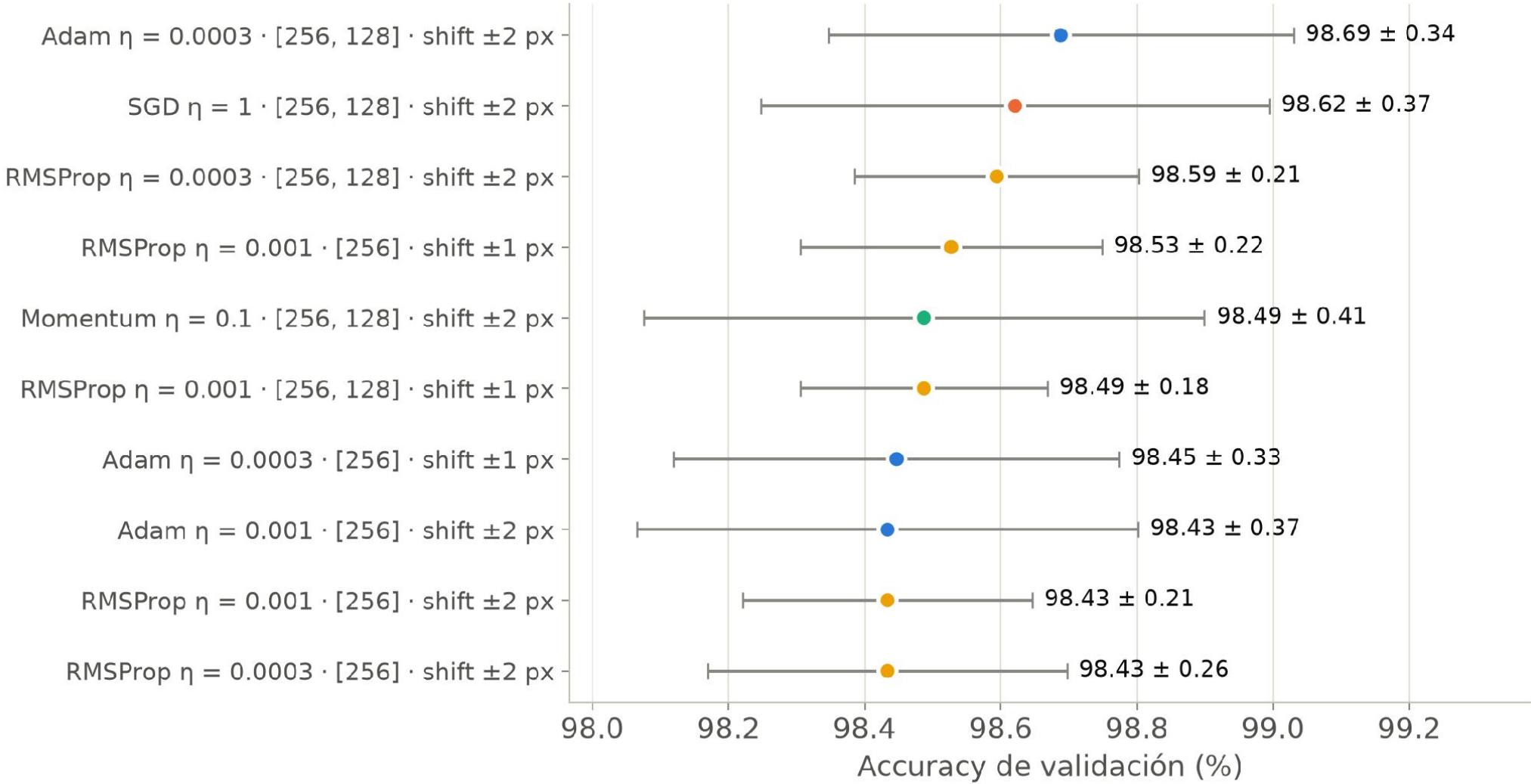
Misma proporción de cada dígito en train y validación

Qué parámetros variamos y como

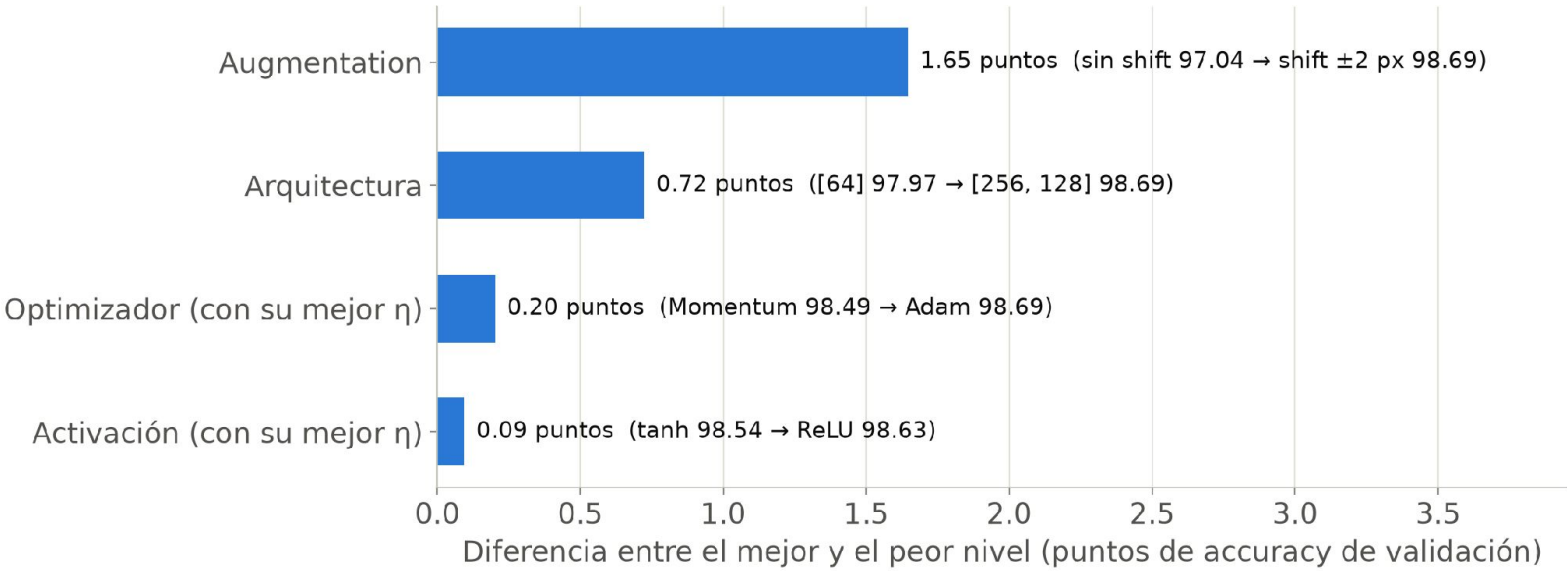


Fijo en todas: tanh · salida logística + MSE · batch 32 · early stopping (paciencia 20)

¿Qué combinación ganó, y qué factor pesa más?



Las 10 mejores de 180 · validación · 3 semillas



Diferencia entre el mejor y el peor nivel de cada factor

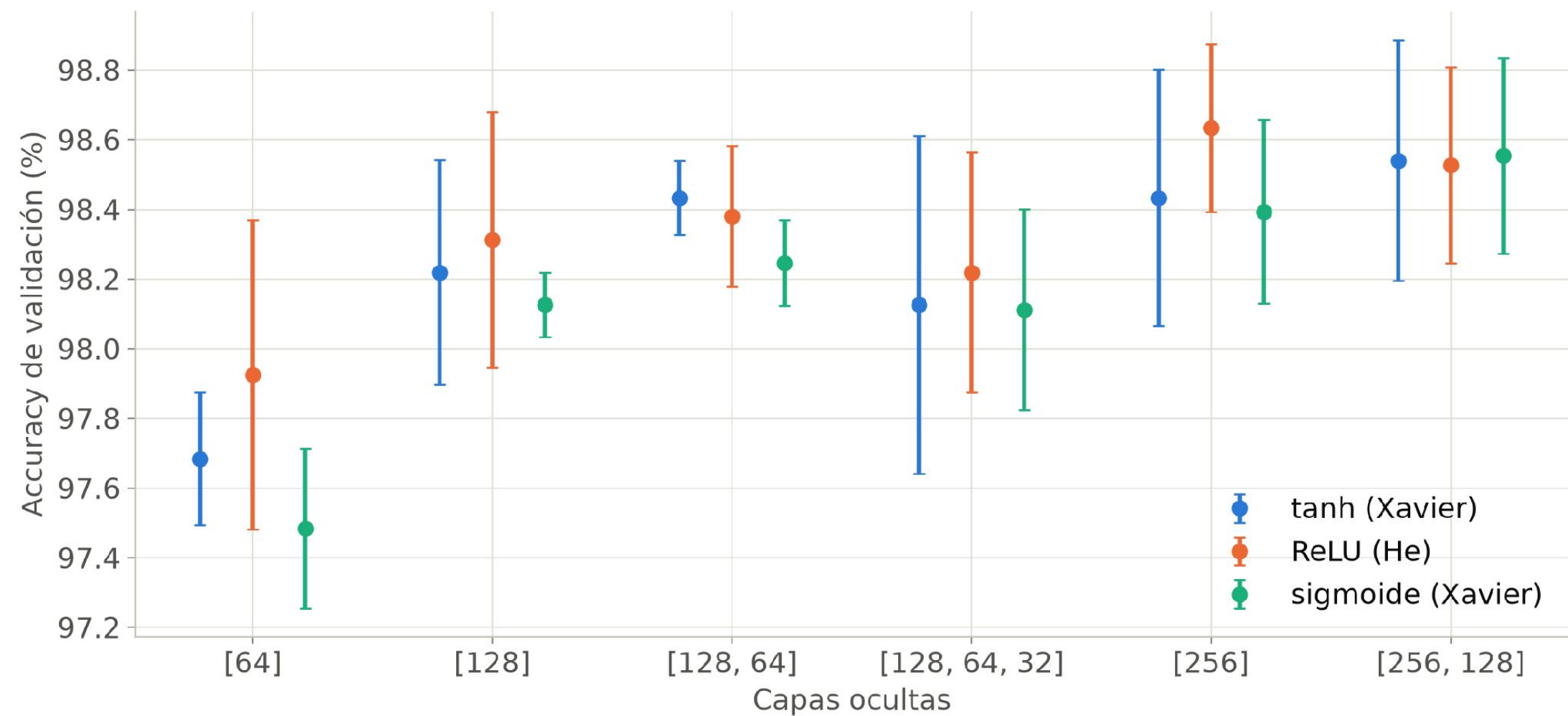
Ganadora

Adam $\eta = 0,0003$, [256, 128] y shift ± 2 :
98,69 $\pm$ 0,34 %.

Empate entre mejores combinaciones

Las 10 mejores están dentro de 0,3 puntos, del orden del desvío entre semillas.

¿Importa la función de activación?



Mejor η de cada activación · Adam · shift ± 2 px · 3 semillas

Mismo techo

tanh 98,54 %, sigmoide 98,55 % y ReLU 98,63 %, todas dentro del desvío.

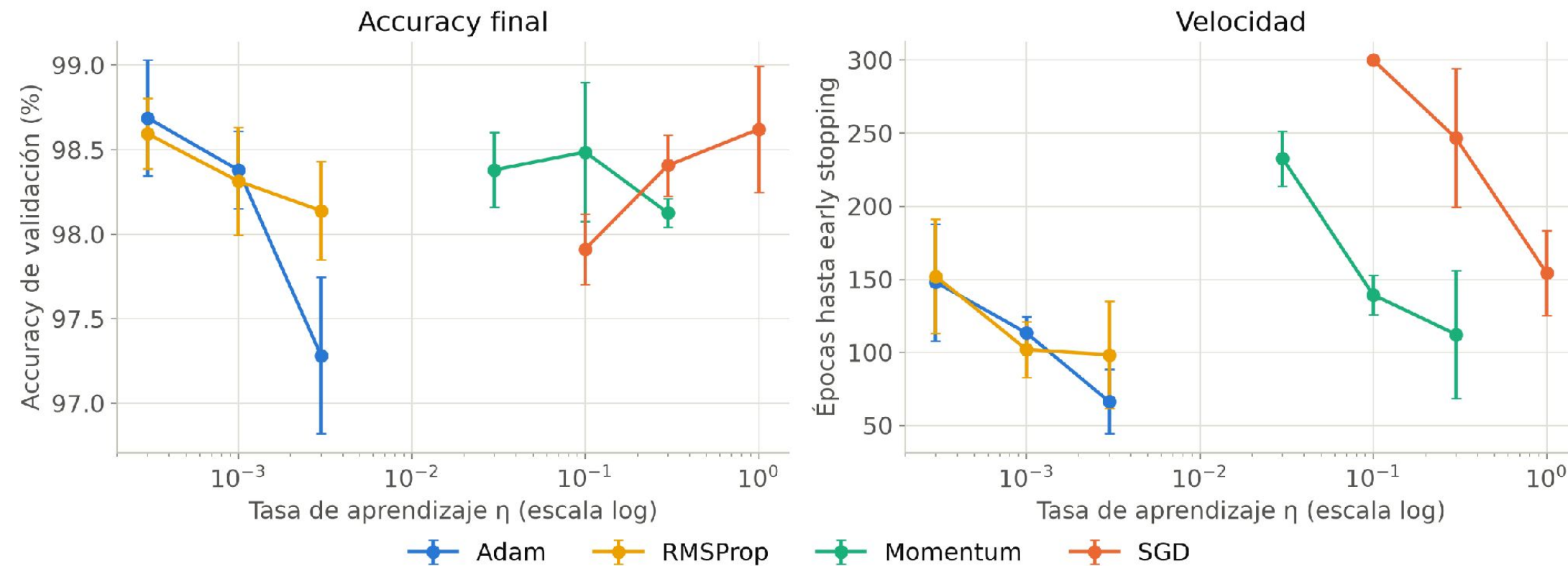
Fijamos tanh

Por eso fijamos tanh en la grilla.

Sí cambia la tolerancia a η

Con $\eta = 0,01$ las tres colapsan y ReLU es la que peor queda.

¿Qué cambia el optimizador?



[256, 128] · shift ± 2 px · tanh · 3 semillas

Cada uno tiene su escala de η

Adam y RMSProp andan en $\sim 10^{-3}$, Momentum en $\sim 10^{-1}$ y SGD en ~ 1 .

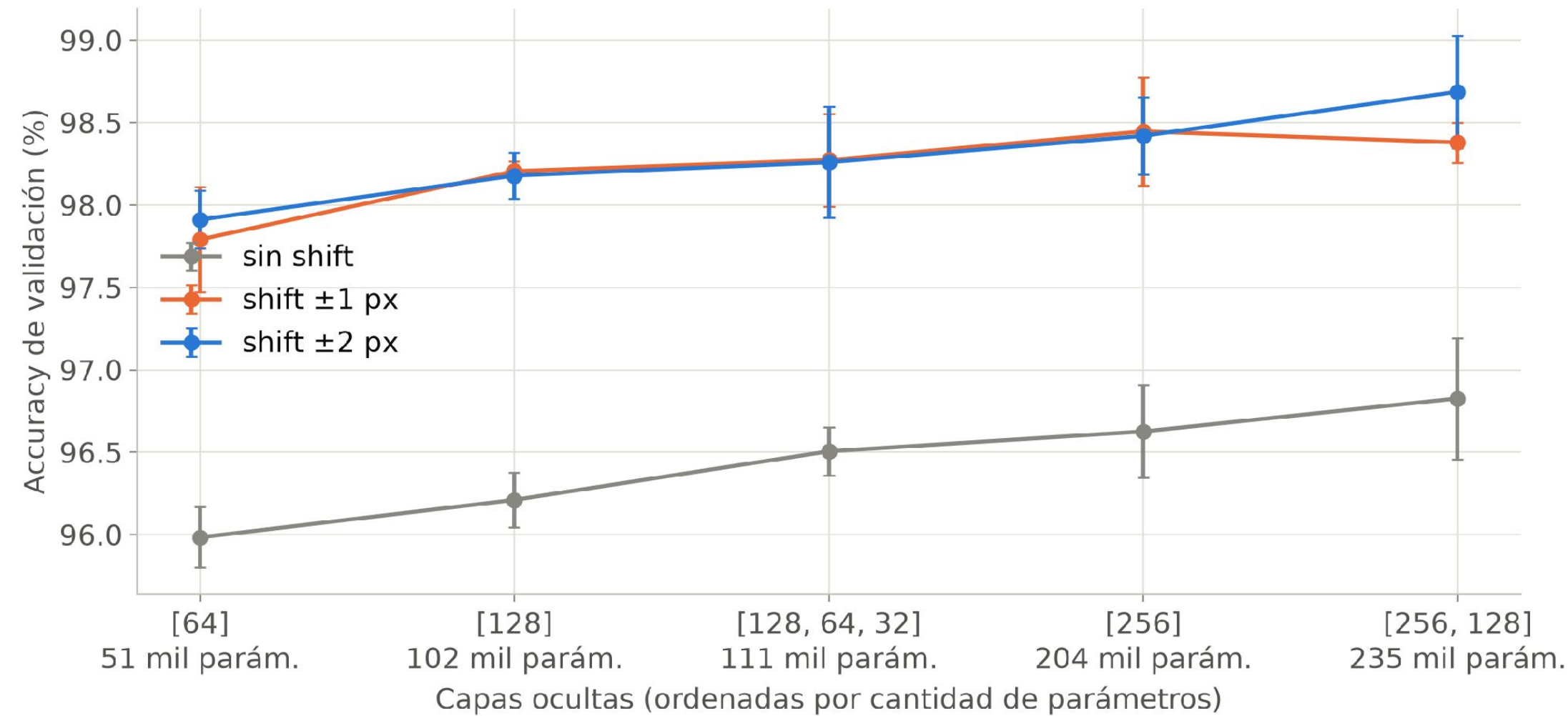
La accuracy resulta similar

Con su mejor η , los cuatro llegan a 98,5–98,7 %.

La velocidad también resulta parecida

En su mejor punto, todos cortan entre 139 y 155 épocas.

¿Ancho o profundidad, con y sin shift?



Adam $\eta = 0,0003$ · tanh · 3 semillas

Más capacidad ayuda poco

Con shift ±2, de [64] a [256, 128] se ganan 0,8 puntos (97,91 → 98,69 %).

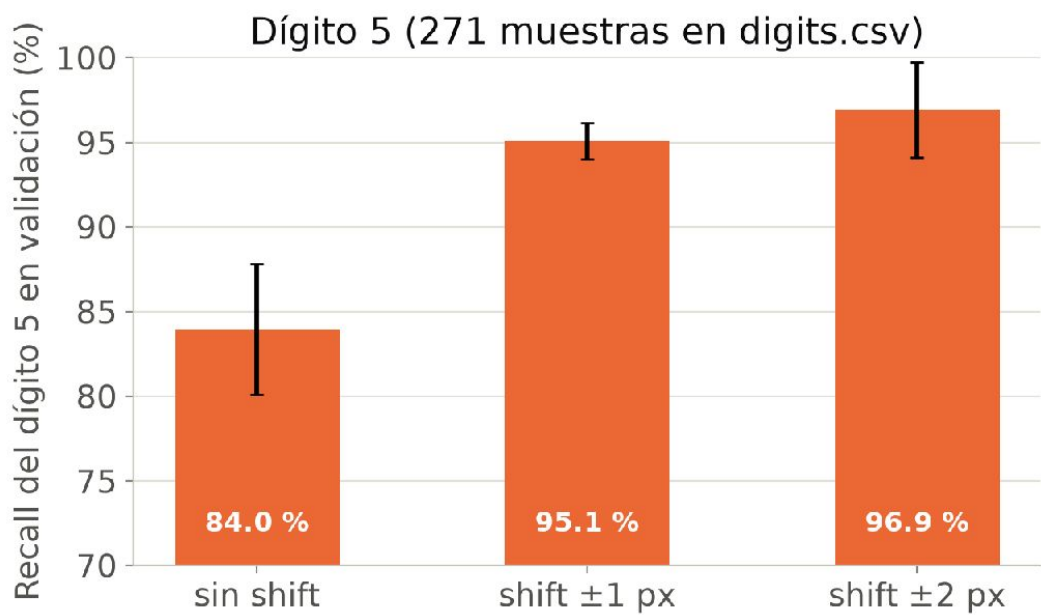
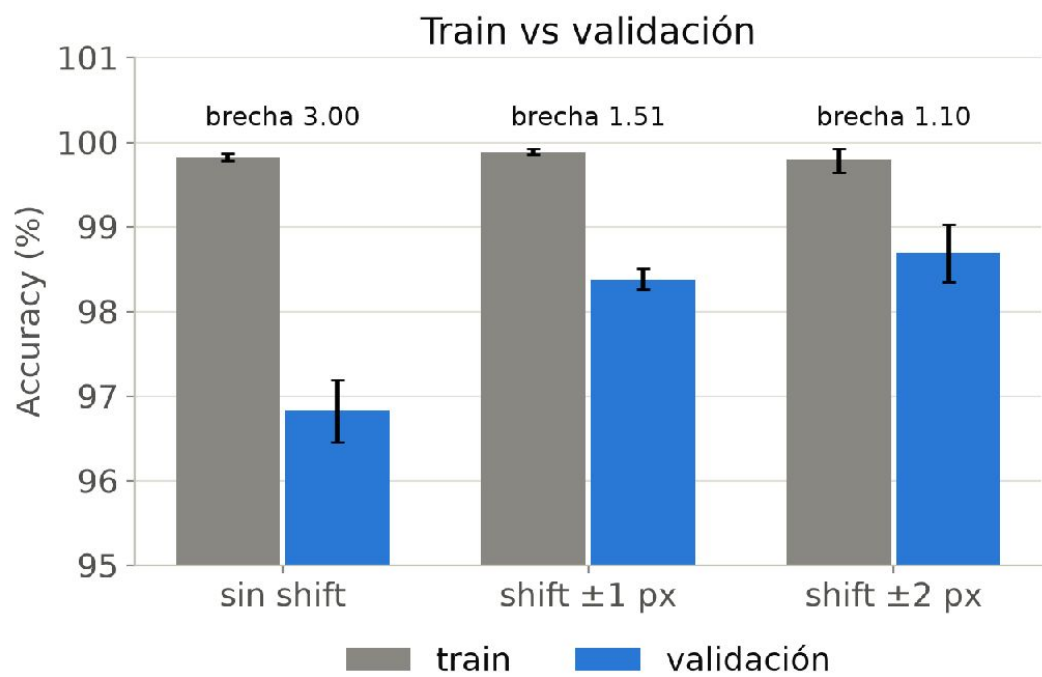
El shift pesa mucho más que la red

En cualquier red suma ~1,7–1,9 puntos. [64] con shift (97,9 %) le gana a [256, 128] sin shift (96,8 %).

Hay interacción

±1 y ±2 empatan en casi todas las redes, pero en [256, 128] gana ±2 (98,69 contra 98,38).

Regularización: achicando la brecha train-validation



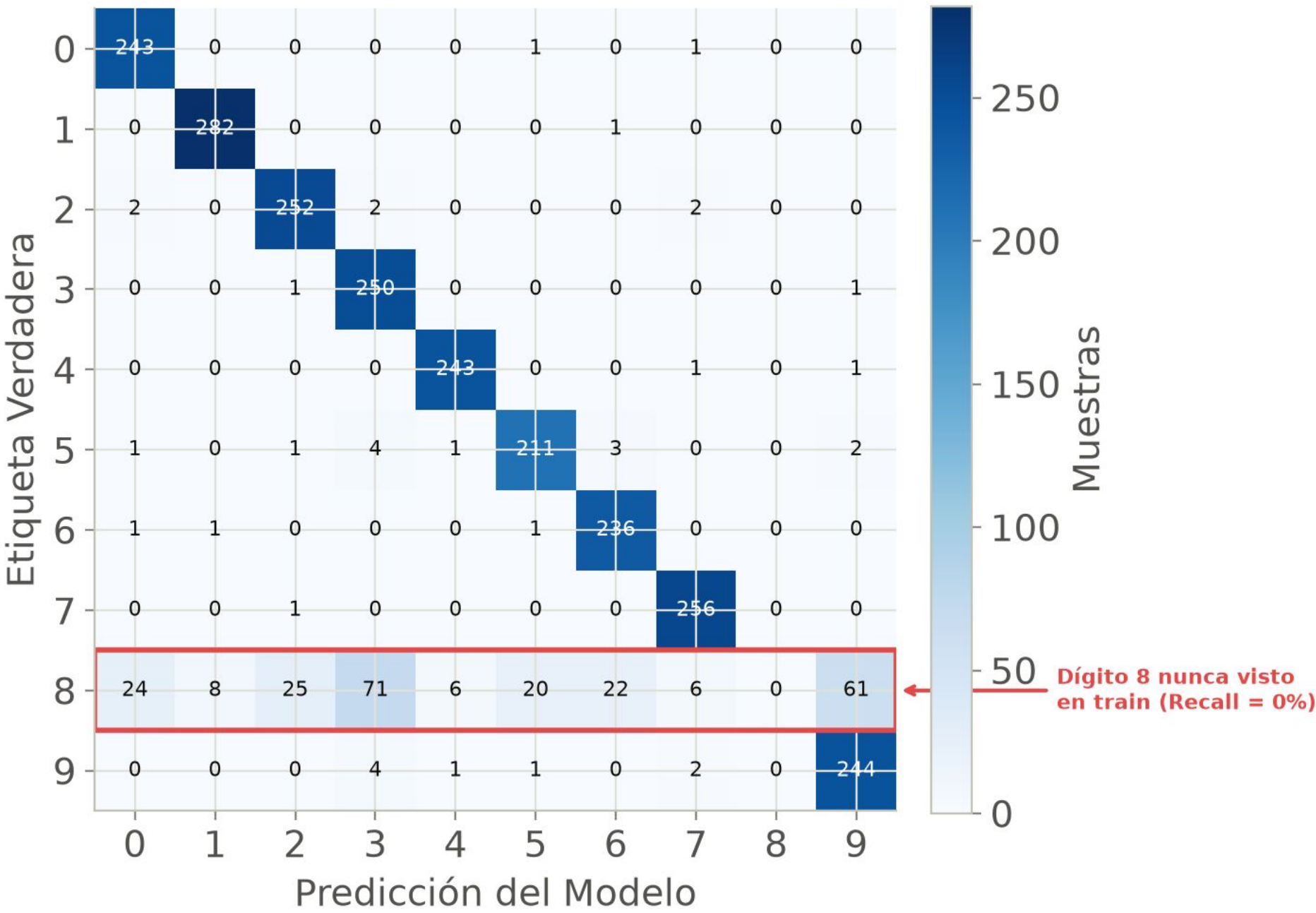
brecha de 3,0 a 1,1

En cada época cada imagen se corre al azar hasta ± 2 px. El MLP no sabe qué píxeles son vecinos: el shift se lo enseña con ejemplos.

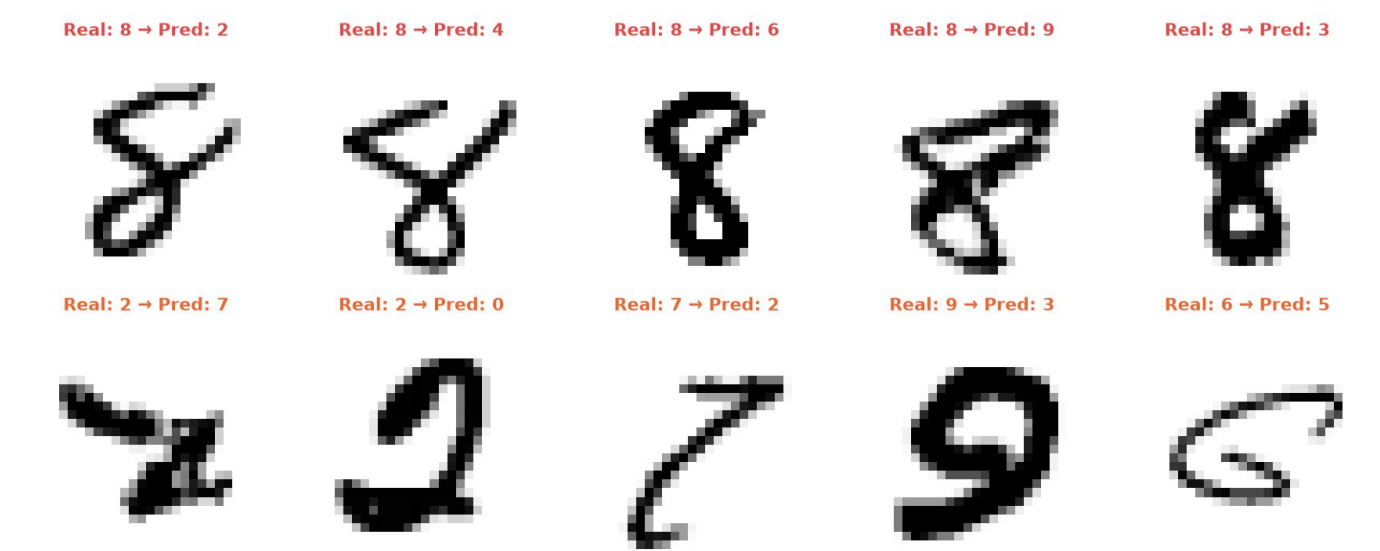
Adam $\eta = 0,0003 \cdot [256, 128] \cdot \tanh \cdot 3$ semillas

	Validación	Brecha train-val	Recall del 5
Sin shift	96,83 %	3,00	84,0 %
Shift ±1 px	98,38 %	1,51	95,1 %
Shift ±2 px	98,69 %	1,10	96,9 %

¿Cómo rinde en datos nunca vistos?



digits_test.csv · 2497 imágenes · semilla 0 · Adam $\eta = 0,0003$ · [256, 128] · shift ± 2 px



Números mal clasificados por el modelo

88,68 %	90,27 %	98,24 %
accuracy en test · $\pm 0,10$ %	techo sin conocer el 8	sobre las clases conocidas

Recall del 8: 0 %. El train no tiene ochos, y la red los confunde con 3, 9 y 2. El resto de los dígitos queda entre 95,4 % y 99,7 %.

Mejorando el Perceptrón Multicapa

Dígitos con más datos: accuracy $\geq$ 98 %

¿Qué trae more_digits?

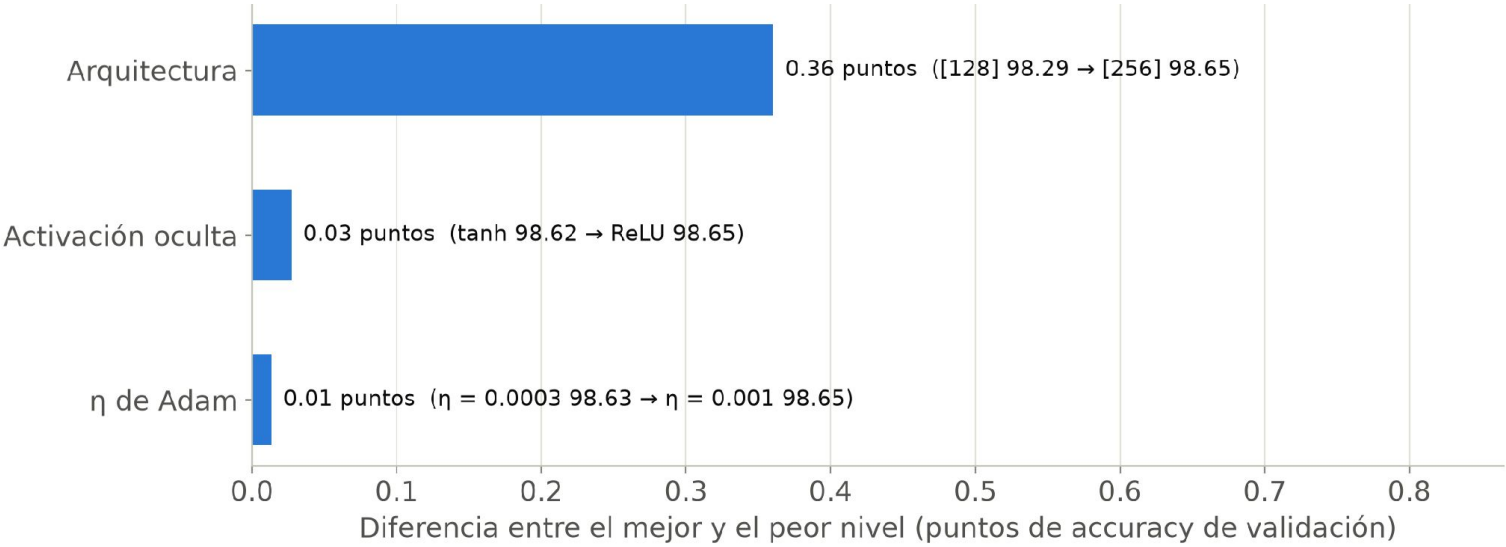
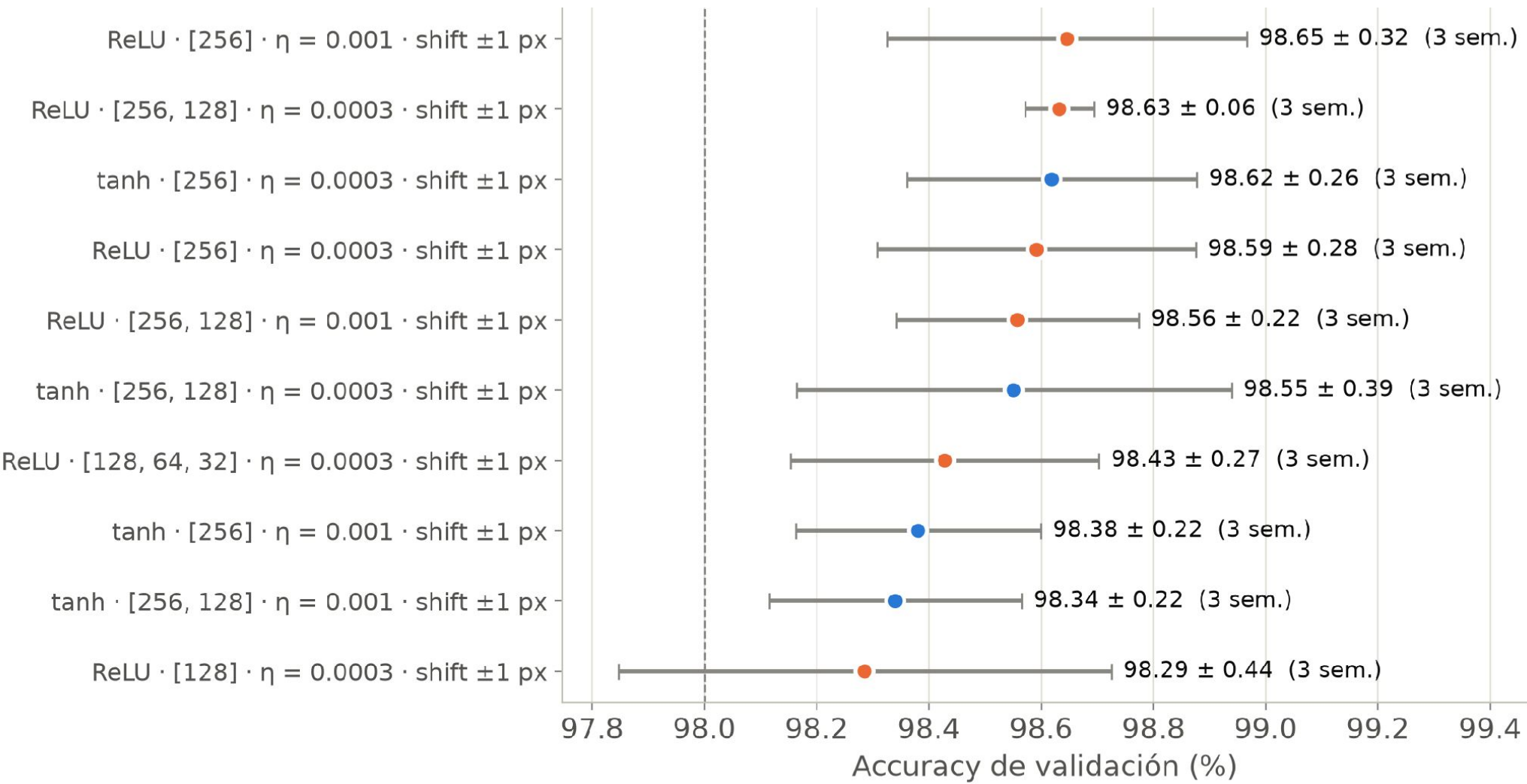
	Imágenes	Ochos	Cincos
digits.csv	12.449	0	271
more_digits.csv	15.741	585	542
Unión sin repetidos	24.501	585	785
digits_test.csv	2497	243	223

3689

imágenes repetidas, eliminadas

¿Se puede mejorar?

Activación (2) × Arquitectura (4) × η de Adam (2) = 16 combinaciones · 3 semillas · shift ± 1

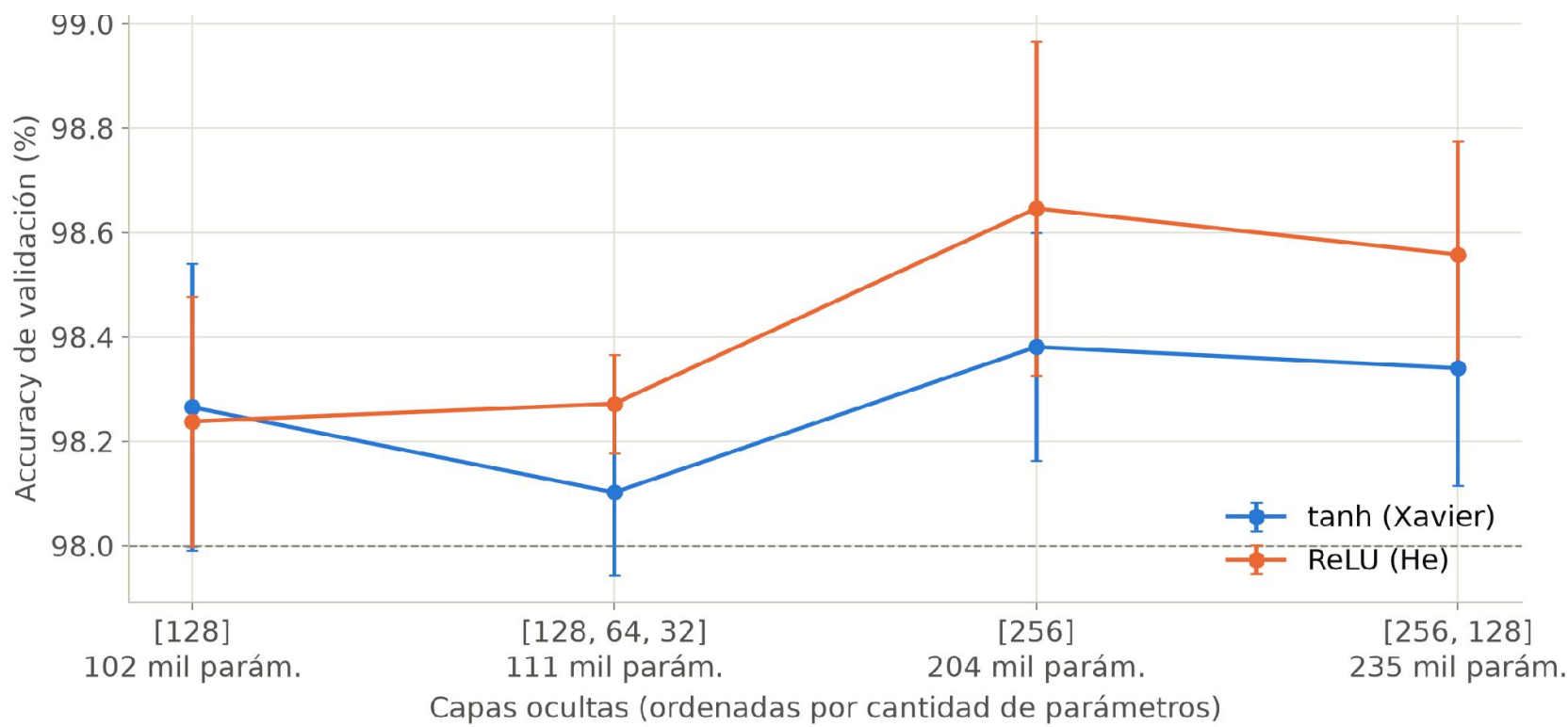


Cuánto mueve cada factor

Las 10 mejores de 16 · validación

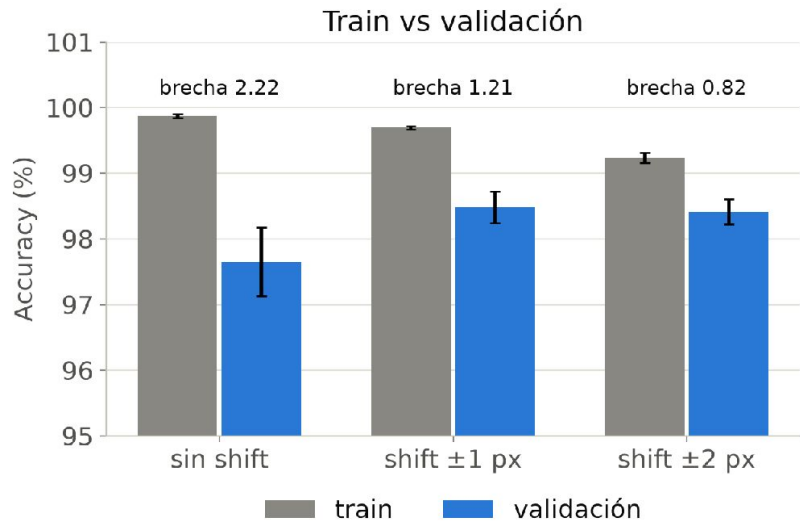
98,65 % ReLU · [256] · $\eta = 0,001$

¿Qué técnicas importan?

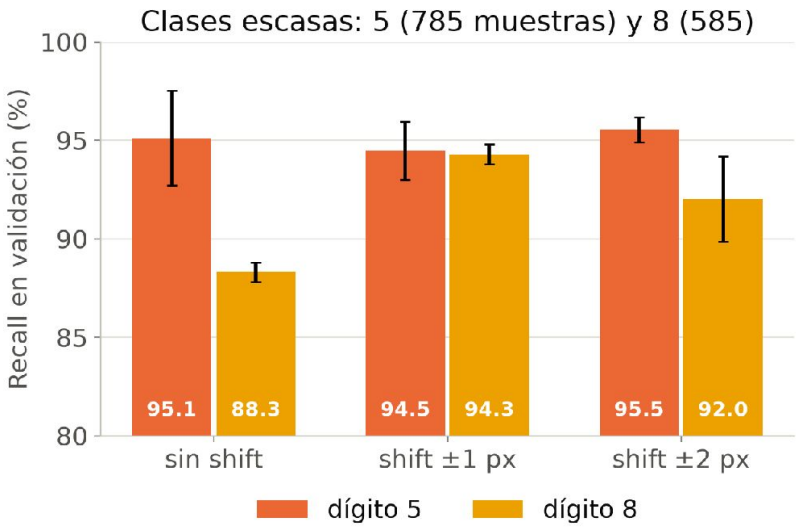


Red × activación · $\eta = 0,001$ · shift ±1

+0,8 puntos
con shift



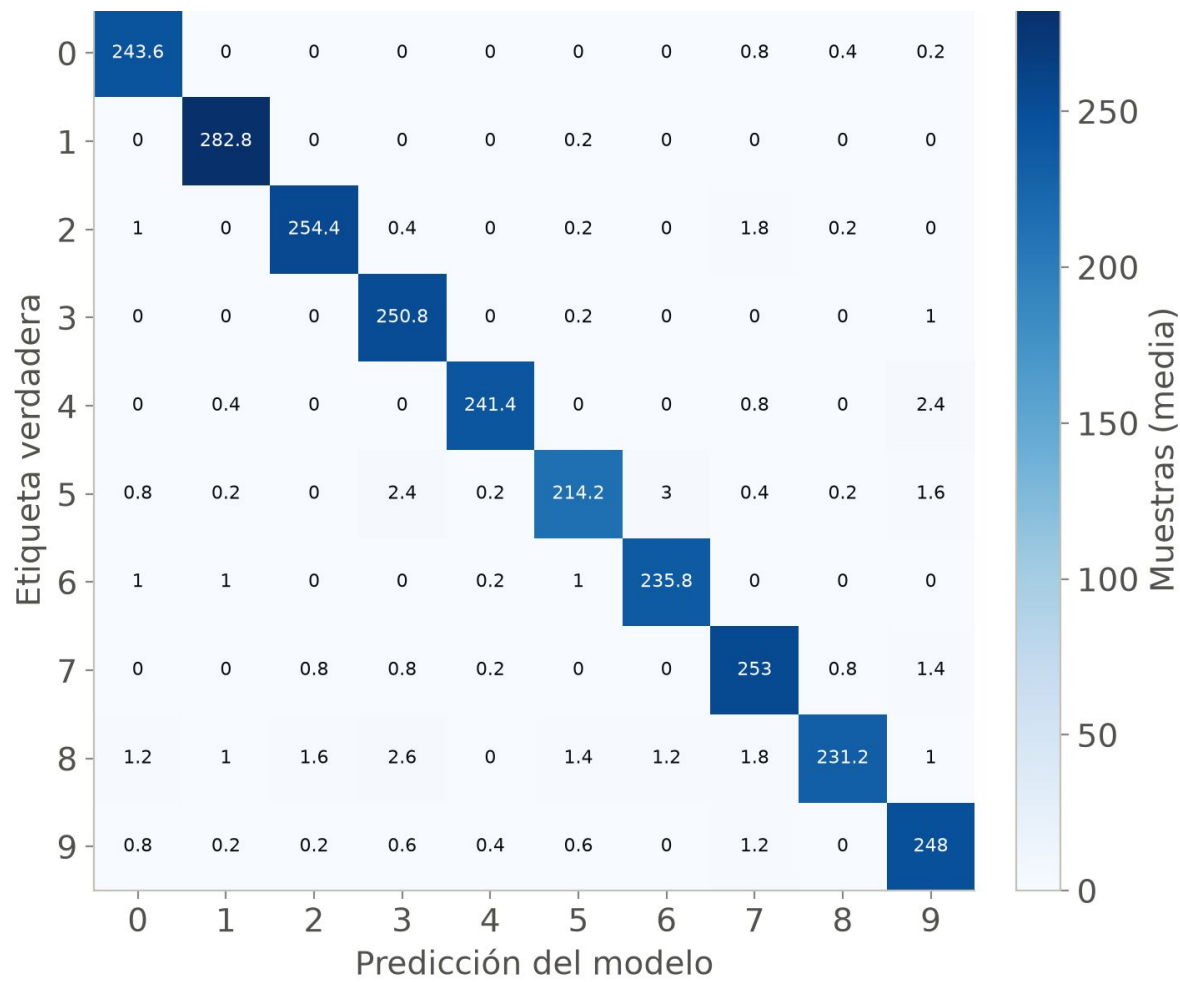
Shift · búsqueda anterior · ReLU [256, 128]



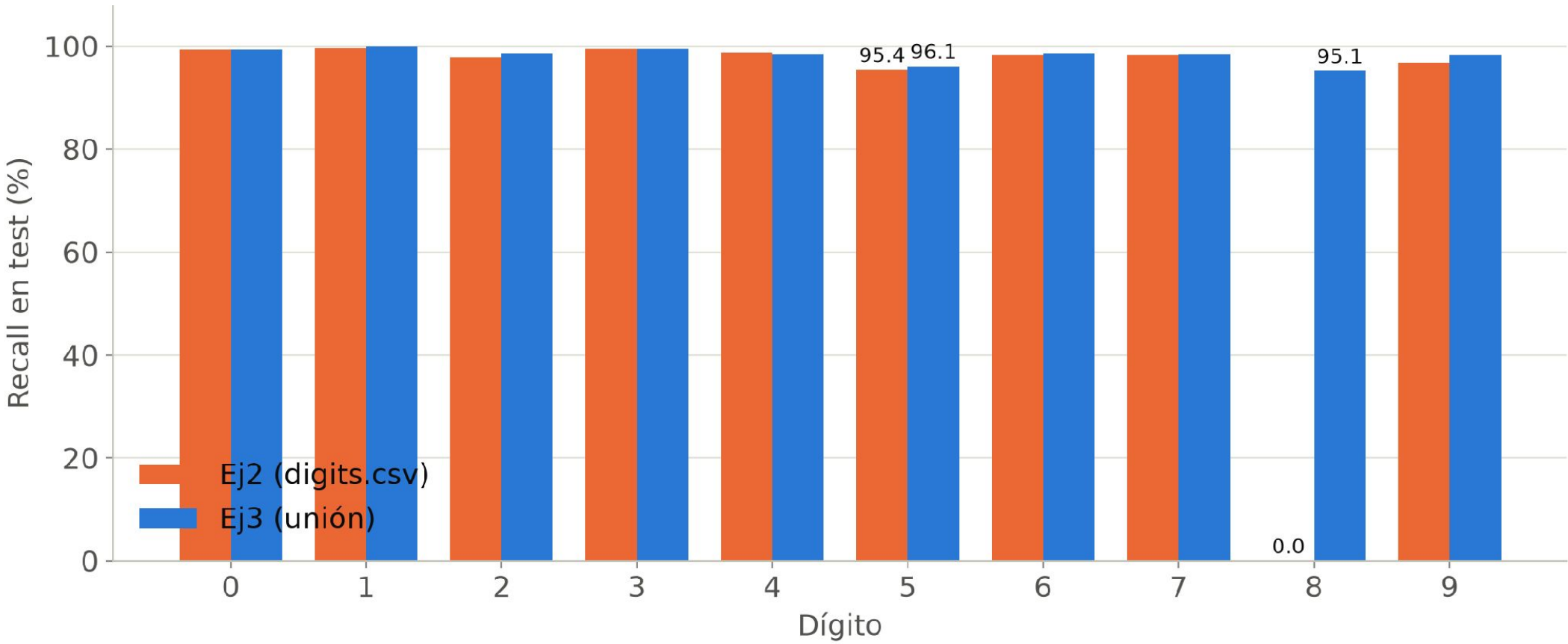
88 → 94 %
recall del 8

0,03
tanh vs ReLU

¿Llegamos al 98 %?



digits_test.csv · media de 5 modelos



Recall por dígito · Ej2 vs Ej3

98,33 %

accuracy en test · ± 0,23

5 de 5

semillas sobre 98 %

+9,26

de los +9,65 puntos vienen del 8

Una conclusión por ejercicio

1

Un TinyModel de 7 parámetros

El perceptrón simple llegó a su máxima capacidad.

2

88,7 % en producción: el límite es el dato

El desplazamiento de la imagen lleva la validación a 98,7 %, pero sin ningún 8 en train el techo en test es 90,3 %.

3

98,3 %: primero el dato, después la técnica

Los 585 ochos explican el 96 % de la mejora.

¿Preguntas?

Grupo 5

Federico Ruckauf

Matías Romanato

Celestino Garrós

Leo Weitz